Net-Gain Scheduling of Parallel Commutators for Rubik's Cubes and Squares up to $N = 20$

Seven Solvers on One Open Engine, from Exact Search to a Size-Generalizing Network

Gaurav
Shiv Nadar University
Greater Noida, India

Shree
Shiv Nadar University
Greater Noida, India

Sunaina
Shiv Nadar University
Greater Noida, India

Kashvi
Shiv Nadar University
Greater Noida, India

Shaurya
Shiv Nadar University
Greater Noida, India

Sharanya
Shiv Nadar University
Greater Noida, India

ABSTRACT

Learned Rubik's Cube solvers stop at the 5×5×5 and train a separate network for every size, while the asymptotically optimal algorithm for large cubes, Demaine et al.'s parallel cluster algorithm, is rarely implemented and measured against other solvers. We study the N×N×1 Rubik's Square and the N×N×N Rubik's Cube for every N from 3 to 20 on one open engine, and run seven solvers on the same seeded instances: IDA* with pattern databases, reduction with Kociemba's two-phase algorithm, Demaine's parallel algorithm, a CayleyPy-style per-size network, two learned cluster-based solvers, and a new method. The new method, Method 5C, keeps Demaine's bulk commutators but schedules them by *net* gain: a bulk move is applied to a whole block of layers when the pieces it fixes per move beat the best single-cluster fix, even if some cells of the block do not need it. Every candidate is applied and checked exactly. Method 5C is the shortest method on every cube from 5×5×5 to 20×20×20 and on the Square from $N = 9$ up; on the 20×20×20 it uses 4,925 moves, 26% fewer than our implementation of Demaine's algorithm and 50% fewer than our staged beam search, and its advantage over Demaine's algorithm grows with N (6% at $N = 6$). Exact search gives the ground truth where it can: God's numbers 6, 10 and 12 for the 3×3×1, 4×4×1 and 5×5×1 by breadth-first search, optimal 5×5×1 solutions by IDA*, and per-size networks that match these optima. A cluster-token transformer trained only on Squares with $N \leq 8$ keeps a correlation of 0.53 with the true scramble depth at $N = 20$. All runs are on CPUs; we release the engine, benchmarks, solvers and every result file.

CCS CONCEPTS

• **Computing methodologies** → **Search methodologies**; *Machine learning*; • **Theory of computation** → Mathematical optimization.

KEYWORDS

Rubik's Cube, heuristic search, parallel algorithms, commutators, beam search, pattern databases, macro-operators, size generalization, Cayley graphs

1 INTRODUCTION

The N×N×N Rubik's Cube is a standard testbed for planning on graphs that are far too large to enumerate. Its worst-case solution length, God's number, is 20 for $N = 3$ [19] and grows as $\Theta(N^2/\log N)$ [9]; finding a shortest solution of a given state is NP-complete [10]. Learned solvers have recently become strong on small cubes. DeepCubeA [1] and EfficientCube [20] solve the 3×3×3, and the CayleyPy approach of Chervov et al. [7] reaches near-optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers. All of these train one network per size on a one-hot encoding of the whole sticker array, so a 6×6×6 needs a new model and a new training run, and none reports a cube beyond $N = 5$.

We ask what it takes to cover the whole range $3 \leq N \leq 20$, for both the cube and its flat relative, the N×N×1 Rubik's Square of Demaine et al. [9], and which approach gives the shortest solutions at each size. The two families share their key structure. Moves never mix *cubie clusters*, clusters of the same type behave identically at every N, and one slice move touches many clusters at once. Demaine et al. use the last fact to solve many clusters with one shared move sequence; that is what makes $\Theta(N^2/\log N)$ achievable. Our contributions:

(1) **An open engine and benchmark** (§4) for both puzzle families and every N. Moves come from 3D geometry rather than hand-written tables; clusters are found automatically and given a sticker order that does not depend on N, so equal cluster types have equal *signatures*. A solvability checker, three move metrics and a harness that re-applies every returned solution make results comparable across methods.
(2) **A full set of baselines on that engine** (§5): IDA* with Korf's pattern databases (Method 1), reduction with Kociemba's two-phase algorithm (Method 2), Demaine's parallel cluster algorithm with 3-cycle batching (Method 3) and a reproduction of the CayleyPy recipe (Method 4). The commutators that Method 2 and Method 3 need are found and verified automatically for every cluster of every N.
(3) **Method 5C, net-gain scheduling of bulk commutators** (§6), our main new method. It keeps Demaine's bulk moves but chooses each one by its net value over a whole block of layers, scores every candidate exactly, shares setup moves and looks one step ahead. It is the shortest method on every cube from $N = 5$ to $N = 20$ and on the Square from $N = 9$, and its lead grows with N.
(4) **Two learned cluster-based solvers**: Method 5B (§7), a staged beam search with exact abstraction tables, small cluster networks and machine-verified commutator actions,

which solved every random state we tested on a two-core CPU; and Method 5A (§8), a cluster-token transformer whose distance estimates carry over to Square sizes it never saw.

(5) **A careful evaluation** (§9) on shared, seeded instances, including exact God's numbers for the smallest Squares, optimal solutions where IDA* finishes, ablations and counting lower bounds. We report where methods fall short, including that exact search does not reach random 3×3×3 states on our hardware and that we could not reproduce CayleyPy's 3×3×3 result without a GPU.

2 RELATED WORK

Structure and complexity. Demaine et al. [9] decompose N×N×1 and N×N×N puzzles into cubie clusters, show that every cluster can be solved without disturbing the others, and obtain $\Theta(N^2/\log N)$ bounds on the diameter by solving many identical clusters with one shared move sequence. Optimal solving is NP-complete for both families [10]. For the 3×3×3 the diameter is exactly 20 in the half-turn metric [19]. Their construction batches clusters that are in the *same* configuration; at practical sizes a cube cluster has about $3 \cdot 10^{15}$ configurations, so exact matches are rare. Method 3 batches 3-cycles instead, and Method 5C relaxes the batching condition further (§6).

Classical solvers. Korf found optimal 3×3×3 solutions with IDA* and pattern databases [16], following Culberson and Schaeffer's pattern databases [8]. Kociemba's two-phase algorithm [14] and its engineered implementation min2phase [5] give near-optimal 3×3×3 solutions in milliseconds. Large cubes are usually solved by *reduction* to a 3×3×3 [23]. Korf's macro-operators [15] learn move sequences that change one part of the state and restore the rest; our verified commutators play the same role inside a learned search. The Kaggle Santa 2023 competition [12] included large cubes; the KMCoders write-up [13] reports, for example, a 33×33×33 in 4,719 moves using cluster-wise 3-cycles applied to many clusters at once, chosen greedily. Their move metric may differ from ours, so we do not compare lengths directly.

Learned solvers. DeepCubeA [1] learns a cost-to-go function by approximate value iteration and searches with batch weighted A*. EfficientCube [20] trains directly on random scrambles from the goal. Q* search [2] scores all children of a node in one network pass and handles large action spaces, including 1,872 meta-actions on the 3×3×3. CayleyPy [7] trains a residual MLP to predict the length of a random walk from the solved state (a diffusion distance) and searches with very wide beams; it reports over 98% optimal 3×3×3 solutions and the first learned 4×4×4 and 5×5×5 solvers, and its authors note that the diameter is unknown for every $N > 3$ [6]. All of these models are trained per puzzle size.

Other uses of the cube. The cube also serves as a benchmark for robotic manipulation [18] and for spatial reasoning in (multimodal) language models [3, 4]; those works use small cubes and do not target solution quality at scale.

Table 1: Cubie cluster types. Wings combine two sticker orbits of one layer. Counts are for one N; $k = \lfloor N/2 \rfloor - 1$.

Puzzle	Cluster type	Stickers	Count
Cube	corners	24	1
	middle edges	24	1 if N odd
	wing edges	48	k
	centers (+, ×, oblique)	24	$((N-2)^2 - [N \text{ odd}])/4$
	fixed centers	6	1 if N odd
Square	corner	16	1
	edge	12	$2k$
	interior	8	k^2
	edge-cross, cross, center	6, 4, 2	odd N only

Size generalization. Networks that read a problem as a set or sequence of parts, such as pointer networks [22] and set transformers [17], can be applied to larger instances than they were trained on. Method 5A applies this idea to cubie clusters.

3 PROBLEM

Puzzles. An N×N×1 Rubik's Square is an $N \times N$ grid of cubies with a top and a bottom colour, plus side stickers on the border. Its $2N$ moves are 180° flips of one row or one column; each move is its own inverse. An N×N×N Rubik's Cube has $6N^2$ stickers and $3N$ slices; counting a quarter, half or three-quarter turn of one slice as one move (the half-turn metric, HTM) gives $9N$ moves. A state is *solved* when every face shows one colour, in any of the reachable orientations (24 for the cube, 4 for the Square). Given a reachable state, the task is to return a short move sequence that solves it.

Metrics. We report lengths after merging commuting same-axis moves (Section 4) in HTM, the quarter-turn metric (QTM, where a half turn counts 2) and the slice-turn metric (STM, where a block of adjacent layers turned together counts 1). For the Square all moves are flips, so HTM = QTM. To compare sizes we also divide lengths by $N^2/\ln N$, the growth rate of the worst case.

Clusters. A cubie can only ever reach the positions in its own cluster. Table 1 lists the cluster types and how many of each an N has; our engine finds them automatically (§4) and the counts are checked by unit tests. A 20×20×20 has 2,400 stickers, 9 wing clusters and 81 center clusters; every one must be solved.

4 ENGINE AND BENCHMARK

All methods run on one engine [11]. Everything below is generic over cuboid dimensions (d_0, d_1, d_2), so the cube (N, N, N) and the Square $(N, N, 1)$ come from the same code.

Moves from geometry. Each sticker stores a doubled, centred cubie coordinate and an outward normal. A move rotates every sticker of one slab by an integer rotation matrix; looking up the rotated (position, normal) pair gives a permutation π, and applying the move is the gather $s \leftarrow s[\pi]$. A slab whose cross-section is not square, such as one row of the Square, only gets 180° turns, and an axis of length 1 gets none. Unit tests check that every move has order 4 (or 2) and that each move followed by its inverse is the identity, for every tested N.

Canonical form. Moves on the same axis commute. We canonicalise a sequence by merging each same-axis run per layer and sorting by layer, and we use the same order to prune search: after a move on axis a, layer ℓ, no move on axis a with layer $\leq \ell$ is allowed. This removes immediate repeats and every redundant ordering of commuting moves.

Clusters with N-invariant signatures. Union-find over all move permutations gives the sticker orbits; orbits sharing cubies are merged into cubie clusters. Inside a cluster, stickers are sorted by face and by the signs and relative magnitudes of their in-face coordinates. This order does not depend on N, so the moves acting on a cluster induce the same local permutations at every N. We hash the local generator set and the solved pattern into a *signature*; tests confirm that, for example, every ×-center cluster of the 5×5×5, 7×7×7 and 9×9×9 has the same signature. A model trained for one cluster type therefore applies at any size.

Projections. Restricting attention to a subset of stickers gives a *projected puzzle*. When it is small we enumerate it by breadth-first search from the solved pattern; the resulting table gives exact distances for that subset and doubles as an exact membership test.

Solvability checker. A state is accepted only if it passes: global and per-cluster colour counts; membership in the exact projection table for every cluster small enough to enumerate (all Square clusters); for cubes, valid corner, middle-edge and wing pieces, corner twist $\equiv 0 \pmod 3$ and, for odd N after normalising orientation by the fixed centers, even middle-edge flip and equal corner and middle-edge permutation parity; and for the Square, joint consistency of each cross cluster with the border clusters it shares moves with. These are necessary conditions. Property tests confirm that the checker accepts scrambled states and rejects a single corner twist, a single middle-edge flip, a swap of two corners on odd cubes and wrong colour counts.

Harness. Every solver implements one interface and returns engine moves. The harness re-applies the moves, decides *solved* itself, and writes one row per instance with lengths in all three metrics, time, nodes, memory, seed, hardware and git commit. A crash or time-out counts as a failure and stays in the denominator.

Shared beam search. Methods 4, 5A and 5B share one beam search. Per step it expands the beam in chunks, scores children with the heuristic, and keeps the W best after removing duplicates by a random 64-bit linear hash (also across the last three steps). Only (score, hash, parent, action) is stored per child; the next beam is rebuilt from parents. Beam states are packed at 3 bits per sticker and can live on the device, in pinned host memory or on disk. The search accepts *macro actions*: fixed move sequences applied as one step and expanded into ordinary moves in the output. A heuristic that can score all children of a parent in one pass is used that way, as in Q* search [2].

Instances. Random states are long random canonical walks (at least 25× the number of moves, and at least 400 moves). This approximates, but does not guarantee, a uniform random reachable state. Every instance is regenerated from a seed and checked by the solvability checker; all methods see the same instances.

5 BASELINES: METHODS 1–4

Pure 3-cycles for any cluster. Method 2 and Method 3 need, for every center and wing cluster of every N, move sequences that 3-cycle three of its pieces and touch nothing else. We find them automatically. For centers we enumerate commutators $[A, B\,C\,B']$ with A and C inner-slice quarter turns on one axis and B an outer face turn; for wings, $[X, Y\,Z\,Y']$ with X an inner slice and Y, Z outer faces. A candidate is kept if, applied to the sticker permutation, it moves exactly three cubies of one cluster. A conjugate $s\,c\,s'$ of a pure 3-cycle c is again pure, so breadth-first search over setup moves s (up to four) extends a few base commutators to every one of the $24 \cdot 23 \cdot 22/3$ 3-cycles of a cluster, each with a short move sequence. A greedy solver then applies, at each step, the 3-cycle with the most pieces fixed per move.

Method 1: IDA with pattern databases.* For the 3×3×3 we use Korf's heuristic [16]: the maximum of a corner database ($8! \cdot 3^7 = 88{,}179{,}840$ entries) and two 6-edge databases ($12!/6! \cdot 2^6 = 42{,}577{,}920$ each), built by breadth-first search over cubie coordinates whose move tables are derived from the engine's sticker permutations (a test checks they agree). IDA* runs in Numba, parallel over the root's moves, at about $1.2 \cdot 10^7$ nodes per second on four cores, with the canonical move order of §4. For the Square the heuristic is the maximum over clusters of each cluster's exact distance, which is admissible because no solution fixes a cluster in fewer moves than it needs alone; since any of the four solved orientations counts, IDA* runs to each of them and keeps the shortest. For the smallest Squares, breadth-first search from all four solved orientations gives the exact distance of every state.

Method 2: reduction with Kociemba's two-phase algorithm. After relabelling colours so that the target is one fixed orientation (for odd N the fixed centers decide), the cube is solved in four stages, each a function from state to moves: (i) every wing cluster with an odd permutation gets one inner-slice quarter turn (Demaine's Lemma 7); (ii) every center cluster is solved with its pure 3-cycles; (iii) corners, middle edges and fixed centers, which behave as a 3×3×3 under outer-face turns, are solved by Kociemba's two-phase algorithm [14]; for even N an odd corner permutation (PLL parity) is fixed by one outer quarter turn first; (iv) every wing cluster is solved with its pure 3-cycles. Outer-face turns keep center clusters on their faces, and pure 3-cycles keep everything else, so the stages never undo each other. On the Square, Method 2 solves the frame and the edge and cross clusters exactly on their projections (Demaine's Lemma 3, $O(N)$ moves) and then each interior cluster with its four commutators $c\,r\,c\,r$ (Lemma 2), at most three each.

Method 3: Demaine's parallel algorithm. Method 3 shares every stage with Method 2 except the batching. On the Square, each interior cluster's optimal word of commutators is read off its exact table; round t applies the t-th commutator of every cluster, and the clusters needing the same commutator are covered by blocks of columns X and rows Y, each fixed by one bulk sequence $C_X R_Y C_X R_Y$ of $2|X| + 2|Y|$ moves (Lemma 4), with Demaine's column groups (Lemmas 5–6). On the cube, batching by identical configuration is useless at practical sizes, so, as the KMCoders write-up suggests [13], we batch 3-cycles. A commutator *type* is an outer-face setup plus a template with layer parameters, such as $[A_i, B\,C_j\,B']$; with layer sets

I and J (all slices of I turned together) it applies the same 3-cycle pattern to every cluster $(i, j) \in I \times J$ for $2|I| + 2|J| + 4$ moves. Each round picks the type with the most pieces fixed per move over all clusters, applies it to every block of clusters that it helps, or applies one single-cluster 3-cycle when that is better.

Method 4: CayleyPy reproduction. Method 4 re-implements the recipe of Chervov et al. [7] on our engine: one residual MLP per puzzle size on one-hot sticker inputs, trained with MSE on fresh non-backtracking random walks to predict walk length, and beam search with hash deduplication for at most 200 steps. Square models have 0.25–0.45M parameters and were trained on 4M ($N \le 5$) or 10M ($N = 6$–8) examples in 1–4 minutes each on CPU. We also trained the paper's 4.0M-parameter 3×3×3 model on 32M examples (77 minutes on CPU), 250× fewer than the paper's 8B.

6 METHOD 5C: NET-GAIN SCHEDULING OF BULK COMMUTATORS

Method 3 applies a commutator type to a block $X \times Y$ of layer pairs only when *every* cell of the block helps the cluster it acts on. Early on many clusters need the same 3-cycle and blocks are large, but the clusters soon diverge, the blocks shrink to single cells and batching fades: on our 20×20×20, 244 of Method 3's 292 center rounds fixed a single cluster. Method 5C changes only this schedule.

Net-gain blocks. For a type t, let $g_t(i, j)$ be the change in the number of solved pieces of the cluster that cell (i, j) acts on; it can be zero or negative. Let λ be the best number of pieces per move that any single-cluster 3-cycle offers in the current state. Method 5C chooses

$$(X^*, Y^*) = \arg\max_{X \cap Y = \emptyset} \sum_{(i,j) \in X \times Y} g_t(i, j) \; - \; \lambda \left(2|X| + 2|Y| + 4 + 2|s_t|\right), \tag{1}$$

where s_t is the type's setup and every cell of $X \times Y$ must be a verified pure 3-cycle. A block is worth applying exactly when its pieces per move beat the best single fix, so cells that do not help, or slightly hurt, are accepted when the block still pays. We maximise by alternation (rows given columns, columns given rows) from the three best rows. Because several cells may act on one cluster, each candidate is then *applied* to the state and scored exactly; this also checks that nothing outside the clusters moved. Wings have one layer parameter and the same rule with Y omitted.

Setup sharing and lookahead. A type whose setup equals the previous step's is charged $2|s_t|$ moves less, because the shared $s'_t\, s_t$ cancels when moves are canonicalised. With one step of lookahead (used for $N \le 15$, where it is affordable), the four best candidates are re-scored by the best step available after them.

Everything else is Method 3. Wing parity, the 3×3×3 stage, the Square frame and edge stages, the type bank and the single-cluster fallback are shared with Method 3, so the difference between the two methods is the schedule alone. On the Square, the net-gain schedule of the interior clusters (gain = drop in exact distance) and Method 3's are both computed, in milliseconds, and the shorter is kept.

Algorithm 1: Method 5C schedule for one family of clusters (centers or wings)

Input: state s; type bank (verified pure 3-cycles per type, cluster and layers); per-cluster 3-cycle libraries
Output: move sequence μ solving every cluster of the family

$\mu \leftarrow [\,]$;
while *some cluster is unsolved* **do**
  $c_1 \leftarrow$ best single-cluster 3-cycle; $\lambda \leftarrow$ pieces fixed per move of c_1;
  for *each of the 24 types with the largest total positive gain* **do**
    $(X, Y) \leftarrow$ alternating maximisation of Eq. 2;
    apply the bulk commutator to s; discard it if anything outside the clusters moved;
    record its exact gain and its moves (less shared setup);
  pick the best gain per move among the blocks and c_1 (optionally after one step of lookahead);
  $s \leftarrow s \cdot \nu$; $\mu \leftarrow \mu \,\|\, \nu$;
return μ;

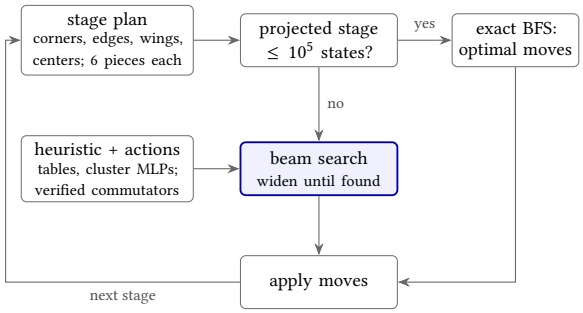


Figure 1: Method 5B. Each stage adds a few pieces to an exact goal. Tiny stages are solved optimally by lookup; the rest by beam search over single moves and verified commutators, guided by exact abstraction tables and small cluster networks.

7 METHOD 5B: CLUSTER-DECOMPOSED STAGED BEAM SEARCH

Method 5B solves the puzzle in a sequence of stages (Figure 1, Algorithm 2). Each stage adds a few pieces to an exact goal: those pieces, and every piece solved by an earlier stage, must show their solved stickers. Each stage is a small search problem with a sharp signal, and later stages cannot silently undo earlier work.

Stage plan. For the Square we follow Demaine et al. [9]: the *frame* (corner, center and edge-cross clusters), then edge clusters two at a time, then cross clusters and interior clusters three at a time, from the outside in. For the cube we use an *edges-first* order: fixed centers, corners, middle edges, wings, then centers, with every wing and center cluster split into stages of 6 pieces. Inner-slice moves never disturb corners, and the commutators below never disturb edges, so the later and more numerous stages have cheap actions that keep earlier work.

Stage actions. A move that touches none of the current targets cannot make progress, so each stage searches only the single moves that touch a target plus the macro actions that do. This cuts the branching factor from $O(N^2)$ macros to a handful. The macro pool is:

- *Square*: for every row r and column c, the commutator $c\,r\,c\,r$, which changes only the cluster at their crossing (Demaine's Lemma 2), and the conjugates $r\,c\,r$ and $c\,r\,c$; $3N^2$ macros.
- *Cube*: $X\,Y\,X'\,Y'$ for an inner-slice quarter turn X and an outer-face quarter turn Y on another axis, which never moves corners; and pure-center commutators $[A, B\,C\,B'] = A\,B\,C\,B'\,A'\,B\,C'\,B'$ with A and C inner-slice quarter turns on one axis (the middle slice included, e.g. $[M, U\,r\,U']$) and B a face turn on another axis. Every candidate is applied to the sticker permutation and kept only if it moves center stickers alone; duplicates with the same permutation are dropped. This leaves 192 pure-center commutators for the 4×4×4 and 29,376 for the 20×20×20, and covers all 81 center clusters of the 20×20×20.

Once every corner and edge is solved, a center stage searches *only* the pure-center commutators, which keep all earlier work by construction.

Heuristics. A stage's heuristic is a weighted sum over clusters:

$$h(s) = \sum_{c \in T} h_c(s) + \lambda_{\text{keep}} \sum_{c \in P} h_c(s) + \lambda_{\text{ex}} \left|\{i \in G : s_i \neq s^*_i\}\right|, \qquad (2)$$

where T are the target clusters, P the earlier clusters that some allowed action can move, G the stage's goal stickers and s^* the solved state ($\lambda_{\text{keep}} = 1$, $\lambda_{\text{ex}} = 0.5$). Each per-cluster term h_c, shared by signature across all N, is one of three kinds:

(1) *Exact table*, when the projected cluster has at most 1.5M states: every Square cluster and the fixed centers.
(2) *Coarse abstraction tables*: the projection after mapping one pair of opposite colours to 1 and the rest to 0. These are pattern databases over a colour abstraction [8] and give admissible lower bounds. They have 2,187 states for corners, 126,720 for middle edges and 735,471 for every center type; wings are skipped because every such projection exceeds the cap. For corners a finer three-class abstraction (one colour to 1, its opposite to 2) has 153,090 states.
(3) *Cluster MLP*: a small residual MLP (hidden width 256, two residual blocks) trained on random walks of the projected cluster to predict walk length, combined with the coarse tables by max. Six such networks (corner, middle edge, wing, three center types) cover every cube size; each trained for 1,500 steps on CPU in 2–14 minutes. A larger set trained on a GPU predicts distance better but searches worse (§9.7).

Corners use the three-class tables alone. The MLP's walk-length targets overestimate the true distance, and taking the max with them flattens the tables' signal; Section 9.6 measures the effect.

Exact small stages. If the projection of a stage's goal stickers under the stage's single moves has at most 100,000 states, we enumerate it and descend greedily on the exact distances, which is optimal for that stage. The Square frame always qualifies: for any N it depends only on rows and columns 0, $\lfloor N/2 \rfloor$ and $N-1$, and is therefore a 3×3×1 with at most 768 states.

Algorithm 2: Method 5B (one state)

Input: state s, stage plan $S_1, \ldots, S_m$
Output: move sequence μ with $s \cdot \mu$ solved

$\mu \leftarrow [\,]$;
for $i \leftarrow 1$ **to** m **do**
  $G \leftarrow \bigcup_{j \le i} \text{goal}(S_j)$;
  if *s already solved on G* **then continue**;
  $A \leftarrow$ single moves and verified macros that move some target of S_i;
  if *projection of G under A has $\le 10^5$ states* **then**
    $v \leftarrow$ optimal moves by greedy descent on its BFS table;
  else
    $h \leftarrow$ stage heuristic (Eq. 1);
    $W \leftarrow W_0$;
    repeat
      $v \leftarrow \textsc{BeamSearch}(s, A, h, W, \text{goal} = G)$;
      $W \leftarrow 4W$;
    until *v found* **or** $W > W_{\max}(|A|)$;
  if *no v* **then return** failure;
  $s \leftarrow s \cdot v$; $\mu \leftarrow \mu \,\|\, v$;
return canonicalise(μ);

Width schedule. Each stage starts at width W_0 (128 for the Square, 512 for the cube) and, if it fails within its step limit (40 and 60 steps), retries at 4× the width up to $W_{\max}$ (2,048 and 8,192). Stages with few actions may widen further while $W \times |\text{actions}| \le 1.2 \cdot 10^6$; the 18-action 3×3×3 corner stage thus reaches 2^{16}.

8 METHOD 5A: ONE NETWORK FOR EVERY SIZE

Method 5A replaces the per-size sticker network with one that reads the puzzle as a set of cluster tokens. Each cluster becomes one token: the one-hot colours of its stickers in the canonical order, projected by a linear map shared by all clusters of the same type, plus a learned embedding of the cluster kind and a small MLP of geometric features (the cluster's two in-face depths divided by $N-1$, and $N/20$). A transformer encoder [21] without positional order mixes the tokens, since they form a set [17]. The value head is *additive*: each token predicts a non-negative cost, the costs are summed, and a small global correction is added. A sum matches the structure of the problem, since total distance grows with the number of unsolved clusters, and it extends to any number of tokens. The per-token costs also serve as per-cluster heuristics inside Method 5B's stages.

Training uses fresh non-backtracking random walks from the solved state on every training size, with the walk length k as target and the loss on $(\hat{k}-k)/(N^2/\ln N)$ so that all sizes contribute on one scale. The model we evaluate has 198,658 parameters (width 64, two layers, four heads) and was trained for 2,500 steps on Squares with $N \in \{4, \ldots, 8\}$ only, in 35 minutes on CPU. At test time it runs full-puzzle beam search for $N \le 7$ and, above that, Method 5B's stages

with its per-token costs as the stage heuristic. The architecture also has a Q*-style move head; we did not evaluate it.

9 EXPERIMENTS

Setup. All runs used CPUs without a GPU. Method 5B, Method 5A and Method 4 ran on a 2-core machine with one thread per process; Methods 1–3 and Method 5C ran on a 4-core machine, one process per run (Method 1's 3×3×3 search uses all four cores). Square experiments use 10 seeded random states for $N \le 12$ and 5 for $13 \le N \le 20$; cube experiments use 2 states for $N \le 10$ and one each for $N = 15$ and $N = 20$. Every method sees the same instances, and every number below comes from the harness's result files, from which our tables and figures are generated by script. Means are over solved instances; a superscript gives the solved fraction when it is below 100%. Times are comparable within a machine only.

9.1 Coverage and the main comparison

Methods 2, 3 and Method 5C solved all Square instances (140/140) and all cube instances (16/16), and Method 5B solved all 140 of 140 Squares and all 16 of 16 cubes (Tables 2 and 3). Which method is shortest depends on the size. On the smallest Squares, exact search and the per-size networks of Method 4 are optimal. From the 9×9×1 on, where no per-size network exists, Method 5C is the shortest method: strictly shorter than Method 3 at 12 of the 12 sizes from 9 to 20 (296 against 304 and 380 moves for Method 3 and Method 2 on the 20×20×1). On the cube, Method 5C is the shortest method at every size from $N = 5$ on. On the 20×20×20 it needs 4,925 moves against 6,612 for Method 3, 8,259 for Method 2 and 9,904 for Method 5B, in 2.6 minutes. Batching alone (Method 3 against Method 2, which share every other stage) saves 20% there; Method 5C's schedule saves another 26% on top. The learned staged search Method 5B is the longest of the four at every cube size: it searches for each stage's moves with learned and abstraction heuristics instead of applying prescribed 3-cycles, and pays for that generality in length (§9.6).

9.2 Where Method 5C's gain comes from

Method 5C's advantage over Method 3 grows with N: 6% at the 6×6×6, 21% at the 10×10×10, 23% at the 15×15×15 and 26% at the 20×20×20. Its length divided by $N^2/\ln N$ rises from 21.7 to 36.9 between $N = 6$ and $N = 20$, against 23.1 to 49.5 for Method 3 (Figure 3). The gain is in the centers, which are 85% of Method 5C's moves on the 20×20×20: there, 229 of its 333 center rounds were bulk moves, against Method 3, where 244 of 292 rounds fixed a single cluster. Table 4 removes the lookahead and the setup sharing. Net-gain blocks alone, without lookahead, already give most of the gain (on the 20×20×20, 4,925 moves against Method 3's 6,612). One step of lookahead shortens solutions by a further 2–8% (4,794 moves on the 20×20×20, our default only up to $N = 15$ because it multiplies the time by about four). Setup sharing matters least: it saved 4% on the 10×10×10, less at $N = 7$ and 8, and nothing on the 6×6×6, where there are few bulk moves to chain.

9.3 Exact optima on small puzzles

Breadth-first search from all four solved orientations visits all 768, 82,944 and 10,616,832 states of the 3×3×1, 4×4×1 and 5×5×1; their diameters, God's numbers in our move set, are 6, 10 and 12 moves. Method 1's IDA* solves random Squares optimally as follows: N=3: 10/10, N=4: 10/10, N=5: 10/10, N=6: 0/10 (120 s per state; 13 s per 5×5×1 on average). At $N = 6$ the max-cluster heuristic is too weak and every search ran out of time, which marks where exact search stops on the Square. Table 5 scores each method against the optimum on the 3×3×1 and 4×4×1. The per-size network of Method 4 found the optimum on every instance (10/10 and 10/10), and on the 5×5×1 it matched Method 1's optimal mean exactly (Table 2). The cluster-at-a-time methods pay for their structure on these small sizes: Methods 2, 3 and Method 5C (identical here, since there is nothing to batch) were 0.6 and 4.0 moves above the optimum, Method 5B 1.2 and 7.0.

On the 3×3×3, Method 1's pattern databases have exactly the sizes given in §5, with largest entries 11 (corners) and 10 (edges) in HTM. On 100 random scrambles of 12 moves, IDA* returned an optimal solution for 91/100 within 120 s each, 13.2 HTM (17.6 QTM) on average with optima between 8 and 16. On 2 random states it found no solution within 33 minutes each, after about $2.7 \cdot 10^{10}$ nodes, proving only that the optimum exceeds 16 moves. The two-phase solver used by Methods 2, 3 and Method 5C returns 21 moves on average in milliseconds.

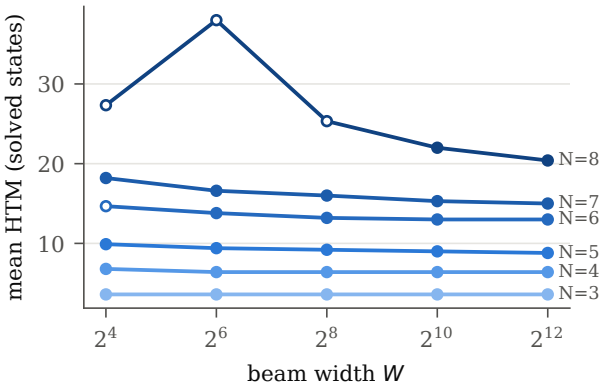


Figure 2: Method 4: mean solution length against beam width W, one per-size network for each Square size $N = 3, \ldots, 8$ (10 states each; means over solved states). Hollow markers: not all 10 states solved, so the mean covers only the easier ones.

9.4 Comparison with per-size networks

Method 4's per-size networks are the strongest method we have on the sizes where they exist. On the 60 Square instances with $3 \le N \le 8$ that both methods solved, Method 5B's solutions were on average 2.35× as long as Method 4's at W=4096 (Table 2). As CayleyPy reports for cubes [7], wider beams help (Figure 2). They also decide coverage at the largest trained size: on the 8×8×1, Method 4 solved 3/10 states at W=16, 9/10 at W=256 and all from W=1024 on, at 17.4 s per state for W=4096. The cost is a separate model and training run for every size; we have them only for $N \le 8$.

Table 2: Rubik's Square, random states: mean HTM of every method on the same instances. "optimal": exact BFS ($N \leq 4$) or Method 1's IDA* ($N = 5$). Method 4: per-size network, W=4096. Bold: shortest method that solved every instance of that size. A superscript gives the solved fraction when below 100%; "–" = not run (no model or out of scope), "$-^{0/k}$" = tried, none solved. Methods 2, 3, 5B and 5C solved every instance.

N	n	optimal	Method 4	Method 5A	Method 5B	Method 2	Method 3	Method 5C
3	10	3.6	**3.6**	–	4.8	4.2	4.2	4.2
4	10	6.4	**6.4**	**6.4**	13.4	10.4	10.4	10.4
5	10	8.8	**8.8**	10.0	22.2	20.7	20.7	20.7
6	10	–	**13.0**	15.0	28.0	30.6	28.8	28.8
7	10	–	**15.0**	$23.5^{4/10}$	41.8	39.9	37.9	37.9
8	10	–	**20.4**	56.4	61.6	58.0	54.4	53.8
9	10	–	–	70.0	83.2	68.4	65.2	**65.0**
10	10	–	–	$85.5^{8/10}$	103.4	94.4	85.6	**84.8**
11	10	–	–	–	121.4	103.2	93.0	**92.8**
12	10	–	–	$-^{0/1}$	143.0	132.0	113.4	**112.8**
13	5	–	–	–	170.4	148.0	132.4	**130.8**
14	5	–	–	–	221.2	180.4	156.4	**155.2**
15	5	–	–	–	242.8	194.6	169.0	**167.4**
16	5	–	–	–	280.0	236.0	196.8	**194.8**
17	5	–	–	–	298.4	247.4	209.0	**206.6**
18	5	–	–	–	354.8	292.4	240.0	**236.8**
19	5	–	–	–	394.4	327.8	271.0	**262.2**
20	5	–	–	–	486.4	379.6	304.0	**295.6**

Table 3: Rubik's Cube, random states: mean HTM of every method on the same instances (bold: shortest), then Method 5C's QTM, its length divided by $N^2/\ln N$ and its time per state on the 4-core machine. Every method solved every instance.

N	n	Method 5B	Method 2	Method 3	Method 5C	Method 5C QTM	Method 5C/ $(N^2/\ln N)$	Method 5C time (s)
3	2	52.0	**21.0**	**21.0**	**21.0**	31.0	2.6	0
4	2	246.0	219.0	**192.5**	196.0	224.5	17.0	1
5	2	452.5	319.5	258.0	**253.0**	281.0	16.3	1
6	2	724.5	574.5	464.5	**437.0**	481.5	21.7	4
7	2	1,045.0	811.0	633.0	**564.0**	614.5	22.4	7
8	2	1,510.5	1,131.0	897.5	**806.5**	863.0	26.2	17
10	2	2,260.5	1,900.5	1,509.0	**1,196.5**	1,256.5	27.6	37
15	1	5,280.0	4,565.0	3,578.0	**2,765.0**	2,927.0	33.3	223
20	1	9,904.0	8,259.0	6,612.0	**4,925.0**	5,188.0	36.9	158

Table 4: Method 5C components on the cube (mean HTM, same instances): Method 3's schedule; Method 5C without lookahead; full Method 5C (lookahead on, the default for $N \leq 15$); Method 5C with lookahead but without setup sharing. "–" = not run.

N	Method 3	no lookahead	with lookahead	no setup sharing
6	464.5	450.5	437.0	436.5
7	633.0	578.0	564.0	570.5
8	897.5	836.0	806.5	810.5
10	1,509.0	1,296.0	1,196.5	1,246.0
15	3,578.0	2,836.0	2,765.0	–
20	6,612.0	4,925.0	4,794.0	–

9.5 Does one network generalise across sizes?

Table 6 shows that the distance estimate of Method 5A does not collapse on sizes it never saw: its correlation with the true walk depth is 0.64 at $N = 8$, the largest training size, 0.66 at $N = 10$ and 0.53 at $N = 20$. These correlations are moderate; a model of 198,658 parameters trained for 2,500 CPU steps is far from converged.

As a solver, Method 5A solved these fractions of instances: N=4: 10/10, N=5: 10/10, N=6: 10/10, N=7: 4/10, N=8: 10/10, N=9: 10/10, N=10: 8/10, N=12: 0/1. With full-puzzle beam search it matched or came close to Method 4's per-size networks for $N \leq 6$, including the optimum on all ten 4×4×1 instances (Table 5), but at $N = 7$ six of ten full-puzzle searches used up their step limit without reaching the goal: the search horizon, not time, was the problem. Inside Method 5B's stages ($N \geq 8$) it solved most states (the two 10×10×1 failures hit the 300 s budget), with solutions shorter than

Table 5: Optimality on the smallest Squares. "Gap" = mean HTM above the exact optimum over solved instances; "optimal" = instances solved in exactly the optimal length. Method 5A was evaluated for $N \geq 4$ only.

Puzzle	God's	mean opt.	Method	solved	gap	optimal
3×3	6	3.6	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	–	–	–
			Method 5B	10/10	1.2	1/10
			Methods 2, 3, 5C	10/10	0.6	5/10
4×4	10	6.4	Method 4 (W=4096)	10/10	0.0	10/10
			Method 5A	10/10	0.0	10/10
			Method 5B	10/10	7.0	0/10
			Methods 2, 3, 5C	10/10	4.0	3/10

Table 6: Method 5A, trained on Squares with $N \in \{4, \ldots, 8\}$ only: agreement of its distance estimate with the true random-walk depth at the end of training, on fresh walks (* = size never seen in training). RMSE is in units of $N^2/\ln N$.

N	4	5	6	7	8	10*	14*	20*
Pearson r	0.48	0.62	0.67	0.61	0.64	0.66	0.65	0.53
normalised RMSE	0.71	0.63	0.58	0.61	0.57	0.57	0.60	0.70

Method 5B's own: on the 62 instances both solved, Method 5B's solutions were 1.65× as long as Method 5A's overall and 1.20× as long on the 18 instances with $N > 8$, sizes outside Method 5A's training range. The cost is time: transformer scoring on CPU made Method 5A's staged solves several times slower than Method 5B's (Table 2). The one 12×12×1 we tried did not finish within 900 s, and we stopped the planned $N = 12$ and $N = 14$ runs there; larger sizes need a GPU or a cheaper scoring path such as the move head.

9.6 Ablations of Method 5B

Table 7 removes one component at a time.

Macro actions are what make large Squares fast. Without them the 8×8×1 still solves at the same length but takes 15 s instead of 0.9 s per state, and at $N = 12$ only 4/5 states solve within 600 s (111 s per state against 4.5 s). A commutator turns a "break, then restore" detour, which the summed heuristic scores as uphill, into one downhill step.

Exact small stages shorten solutions rather than speed them up: solving the frame optimally instead of by beam search gives 59 instead of 67 moves at $N = 8$ and 144 instead of 175 at $N = 12$.

On the cube, *restricting late center stages to pure-center commutators* left lengths unchanged here and saved time (74 s against 133 s per 5×5×5 state). The *corner heuristic* result is mixed: taking max(MLP, tables) instead of the tables alone made 4×4×4 solutions longer (296 against 246 moves) and 5×5×5 solutions shorter (421 against 452), and was slower on both. Removing the coarse tables, so that corners fall back to the MLP alone, gave the same pattern (296 and 436 moves). With two states per size we read these cube differences as indicative only; the clearest cube effect is speed.

9.7 CPU-trained versus GPU-trained cluster networks

Method 5B's cluster networks (§7) were trained twice. The *CPU-trained* set, used for every other Method 5B result in this paper, has width 256 (0.31M parameters per network) and was trained for 1,500 steps per cluster type on a two-core CPU (2–14 minutes each). The *GPU-trained* set has width 512 (1.14M parameters) and was trained for 20,000 steps per cluster type on a Colab T4 GPU (4–5 minutes each), with the same architecture (two residual blocks), data generator, walk lengths, loss and optimiser. The two sets therefore differ in width and in training length, not only in hardware: the comparison below is between two trained model sets, and the GPU is what made the larger one affordable. Both sets are combined with the same exact coarse tables by max, and corners use the tables alone in both, so the corner networks do not affect search.

As predictors, the GPU-trained networks are better for every cluster type (Table 8): their estimates rank random-walk states closer to their true depth, their training loss is lower, and they overestimate less near the goal. As search heuristics they are worse (Table 9). With everything else fixed, Method 5B's solutions were longer with the GPU-trained networks on 5/8 cube states (on average 275.0 against 246.0 moves on the 4×4×4 and 821.5 against 724.5 on the 6×6×6), shorter only on the 5×5×5 (433.0 against 452.5), and each solve took 1.5 to 23 times as long on average per size. The Colab run of the same networks on a T4 confirms the picture: on 14 cube states up to the 10×10×10, its solutions were longer than those of the CPU-trained networks on all 12 states it solved, and 2 states failed when a wing stage exhausted its widest beam. We therefore keep the CPU-trained networks. A network that fits walk length more closely is not a better beam-search heuristic here, which agrees with the corner ablation above: what the search needs is a signal that ranks the few children of each beam step correctly near the goal, and walk length, which overestimates the true distance, is only a proxy for it. With two states per size this is a consistent but small-sample result.

9.8 Scaling with N and lower bounds

Figure 3 divides each mean length by $N^2/\ln N$. Every method's ratio still rises over the tested range, so none is shown to grow at the $N^2/\log N$ rate that Demaine's construction achieves asymptotically. The rise is steepest for Method 5B, which solves clusters a few at a time, and flattest for Method 5C, which shares the most moves between clusters: on the Square, Method 5B goes from 0.59 at $N = 3$ to 3.64 at $N = 20$ and Method 5C reaches 2.21; on the cube, Method 5C's ratio at $N = 20$ is half of Method 5B's.

Table 10 puts the counting bounds next to our lengths. The bounds only say that *some* state needs that many moves, so they cannot certify optimality of any single solution, but on the cube even Method 5C's lengths are about an order of magnitude above them, so either the bounds are weak or there is much room left; both are likely.

10 LIMITATIONS

- *Small samples and CPU only.* We test few states per size, especially for large cubes (one each at $N = 15$ and 20), so the large-N comparisons rest on single instances; the gaps

Table 7: Ablations of Method 5B, removing one component at a time: macro actions, exact small stages, the restriction of late center stages to pure-center commutators, the tables-only corner heuristic, and the coarse abstraction tables (corners then use the MLP alone). Each cell: solved / tried, mean HTM over solved states, mean seconds per state (failures included in time). Square: 5 states per size, 600 s budget; cube: 2 states per size, 300 s budget. The same instances as Tables 2 and 3.

Square	$N = 8$			$N = 12$		
	solved	HTM	s	solved	HTM	s
full Method 5B	5/5	59	0.9	5/5	144	4.5
− macros	5/5	59	15	4/5	136	111
− exact stages	5/5	67	0.5	5/5	175	4.0

Cube	$N = 4$			$N = 5$		
	solved	HTM	s	solved	HTM	s
full Method 5B	2/2	246	31	2/2	452	74
− center restriction	2/2	246	41	2/2	452	133
corners: max(MLP, tab.)	2/2	296	89	2/2	421	125
− coarse tables	2/2	296	87	2/2	436	120

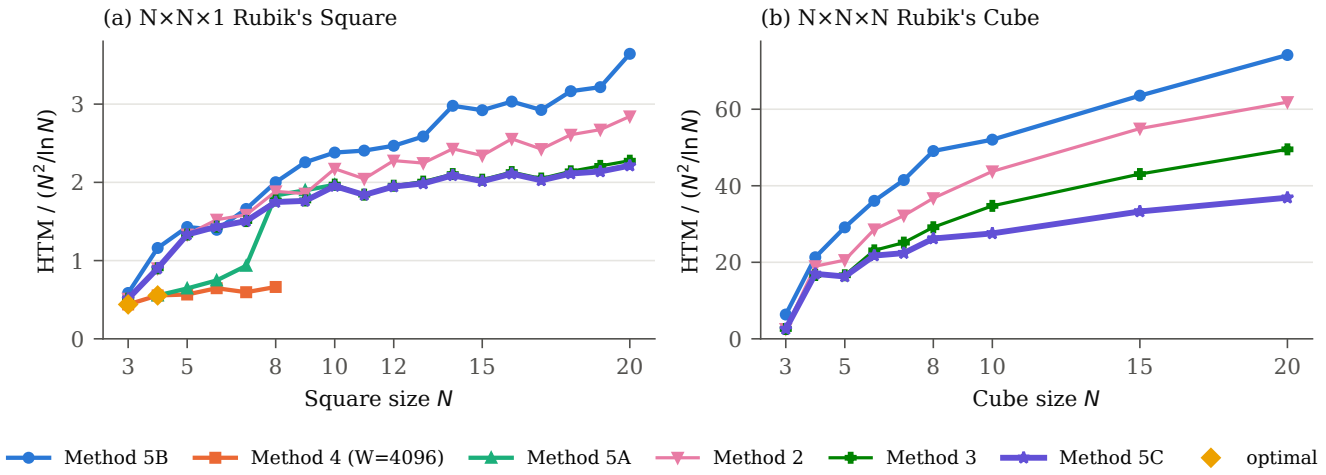


Figure 3: Solution length divided by $N^2/\ln N$, the growth rate of the worst case, for the Square (a) and the cube (b). A flat line would mean growth at the rate of the worst case. Means over solved states; the same instances as Tables 2 and 3.

Table 8: The two sets of Method 5B cluster networks as distance predictors, on fresh random walks of each projected cluster (seeded). ρ: Spearman correlation of the estimate with the walk depth (higher is better); "excess": mean amount by which the estimate exceeds the exact coarse-table lower bound on states at depth ≤ 5 (lower means less overestimation near the goal); loss: final training MSE. †Not used in search (corners use the tables alone).

	ρ		excess		loss	
Cluster	CPU	GPU	CPU	GPU	CPU	GPU
corner†	0.57	0.65	2.6	1.3	42.0	36.2
middle edge	0.67	0.74	2.1	1.5	33.2	28.0
wing	0.75	0.79	3.4	3.3	48.4	39.9
×-center	0.80	0.84	2.5	2.3	23.4	19.6
+-center	0.84	0.87	2.2	2.1	20.2	17.9
oblique center	0.82	0.84	1.9	1.7	19.8	19.4

there are wide, but they are not tight estimates. Timings come from two different CPUs.

Table 9: Method 5B with each set of cluster networks on the same cube instances, the same CPU, the same code and the same 1,800 s budget; only the networks differ. Mean HTM over solved states and mean seconds per state.

N	HTM (CPU-trained)	HTM (GPU-trained)	solved (GPU)	s (CPU)	s (GPU)
3	52.0	56.0	2/2	11	16
4	246.0	275.0	2/2	16	50
5	452.5	433.0	2/2	37	866
6	724.5	821.5	2/2	66	182

- *Still above Demaine's rate.* Every method's length divided by $N^2/\ln N$ still rises over the tested range, Method 5C's most slowly. We have not shown that any of them reaches the $\Theta(N^2/\log N)$ rate at these sizes.
- *Method 5C is greedy.* Its schedule looks at most one step ahead and treats which identical center sticker goes where as fixed; choosing those assignments so that clusters need the same next 3-cycle, and a learned cost-to-go in place of the lookahead, are the obvious next steps. On the 4×4×4,

Table 10: Counting lower bounds on the worst case (God's number) in our move sets (2N Square flips, 9N cube half-turn moves), from the number of reachable center-cluster configurations as in Demaine et al. [9], next to the mean lengths of Method 5B and Method 5C on random states. The bounds are for the worst state, not for these instances.

	Square			Cube		
N	bound	Method 5B	Method 5C	bound	Method 5B	Method 5C
6	2	28	29	35	724	437
8	5	62	54	75	1,510	806
10	9	103	85	126	2,260	1,196
15	18	243	167	262	5,280	2,765
20	39	486	296	557	9,904	4,925

with one center and one wing cluster, there is nothing to batch and Method 5C is not shorter than Method 3.

- *Exact search on the 3×3×3.* With Korf's three databases on four cores, Method 1 solves scrambles of 12 moves optimally but no random state within 30 minutes; larger pattern databases or a GPU would be needed for the standard benchmark.
- *Method 4 on cubes.* With 250× less training data than the original, our 3×3×3 model solved short scrambles but none of 10 random states at $W \in \{256, 1024, 4096\}$ and none of 4 at $W = 16{,}384$ (from the project's status log). This is a statement about our compute, not about the method.
- *Method 5A's evidence is preliminary.* One small Square model, a CPU budget, and no cube model.
- *Checker and sampling.* The checker enforces necessary conditions only, and random states come from long random walks rather than an exact uniform sampler.
- *Missing external baseline.* We did not run the dwalton76 NxNxN solver [23] on our instances.

11 CONCLUSION

Decomposing Rubik's puzzles into clusters with N-invariant signatures lets one engine run seven solvers on the same instances, from exact search to learned heuristics, for every size from 3 to 20. Exact search settles the smallest Squares and shows how quickly it stops scaling; per-size networks match the optimum where they exist; learned cluster-based search covers every size, and a cluster-token network carries its distance estimates to sizes it never saw. At large N the winner is structural: Demaine's idea of fixing many clusters with one shared move sequence. Scheduling those shared moves by net gain rather than by unanimous need, Method 5C solved the 20×20×20 in 4,925 moves, 26% fewer than Demaine's algorithm with 3-cycle batching and 50% fewer than our staged beam search, with a lead that grows with N. Learning that schedule is the natural next step, and the place where the two lines of this work meet. Code, data and result files are available at https://github.com/gauravrai1704/Rubik.

ACKNOWLEDGMENTS

This work was carried out as a course project at Shiv Nadar University.

REFERENCES

[1] Forest Agostinelli, Stephen McAleer, Alexander Shmakov, and Pierre Baldi. 2019. Solving the Rubik's Cube with Deep Reinforcement Learning and Search. *Nature Machine Intelligence* 1, 8 (2019), 356–363. https://doi.org/10.1038/s42256-019-0070-z

[2] Forest Agostinelli, Shahaf S. Shperberg, Alexander Shmakov, Stephen McAleer, Roy Fox, and Pierre Baldi. 2024. A* Search Without Expansions: Learning Heuristic Functions with Deep Q-Networks. arXiv:2102.04518 [cs.AI]

[3] Dhruv Anand and Ehsan Shareghi. 2025. Cube Bench: A Benchmark for Spatial Visual Reasoning in MLLMs. arXiv:2512.20595

[4] Huan ang Gao, Zikang Zhang, Tianwei Luo, Kaisen Yang, Xinzhe Juan, Jiahao Qiu, Tianxing Chen, Bingxiang He, Hao Zhao, Hao Zhou, Shilong Liu, and Mengdi Wang. 2025. CubeBench: Diagnosing Interactive, Long-Horizon Spatial Reasoning Under Partial Observations. arXiv:2512.23328

[5] Shuang Chen. [n. d.]. min2phase: An Implementation of Kociemba's Two-Phase Algorithm. https://github.com/cs0x7f/min2phase. Accessed October 2026.

[6] Alexander Chervov, Dmytro Fedoriaka, Mark Obozov, Elena Konstantinova, et al. 2025. CayleyPy Growth: Efficient Growth Computations and Hundreds of New Conjectures on Cayley Graphs. arXiv:2509.19162

[7] Alexander Chervov, Kirill Khoruzhii, Nikita Bukhal, Jalal Naghiyev, Vladislav Zamkovoy, Ivan Koltsov, Lyudmila Cheldieva, Arsenii Sychev, Arsenii Lenin, Mark Obozov, Egor Urvanov, and Alexey Romanov. 2025. A Machine Learning Approach That Beats Rubik's Cubes. In *Advances in Neural Information Processing Systems (NeurIPS)*. Spotlight. arXiv:2502.13266.

[8] Joseph C. Culberson and Jonathan Schaeffer. 1998. Pattern Databases. *Computational Intelligence* 14, 3 (1998), 318–334.

[9] Erik D. Demaine, Martin L. Demaine, Sarah Eisenstat, Anna Lubiw, and Andrew Winslow. 2011. Algorithms for Solving Rubik's Cubes. In *Proceedings of the 19th Annual European Symposium on Algorithms (ESA 2011) (Lecture Notes in Computer Science, Vol. 6942)*. Springer, 689–700. Full version: arXiv:1106.5736.

[10] Erik D. Demaine, Sarah Eisenstat, and Mikhail Rudoy. 2018. Solving the Rubik's Cube Optimally is NP-complete. In *35th Symposium on Theoretical Aspects of Computer Science (STACS 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 96)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 24:1–24:13. https://doi.org/10.4230/LIPIcs.STACS.2018.24

[11] Gaurav, Shree, Sunaina, Kashvi, Shaurya, and Sharanya. 2026. Rubik: Six Solvers for N×N×1 and N×N×N Rubik's Puzzles on One Engine. https://github.com/gauravrai1704/Rubik.

[12] Kaggle. 2023. Santa 2023 – The Polytope Permutation Puzzle. https://www.kaggle.com/competitions/santa-2023.

[13] KMCoders. 2024. Kaggle Santa 2023 – The Polytope Permutation Puzzle: KMCoders' Solution. https://github.com/wata-orz/santa2023_permutation_puzzle.

[14] Herbert Kociemba. [n. d.]. The Two-Phase Algorithm. https://kociemba.org/cube.htm. Accessed October 2026.

[15] Richard E. Korf. 1985. Macro-Operators: A Weak Method for Learning. *Artificial Intelligence* 26, 1 (1985), 35–77.

[16] Richard E. Korf. 1997. Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. 700–705.

[17] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set Transformer: A Framework for Attention-Based Permutation-Invariant Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

[18] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. 2019. Solving Rubik's Cube with a Robot Hand. arXiv:1910.07113

[19] Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge. 2014. The Diameter of the Rubik's Cube Group Is Twenty. *SIAM Rev.* 56, 4 (2014), 645–670. https://doi.org/10.1137/14097375X

[20] Kyo Takano. 2023. Self-Supervision Is All You Need for Solving Rubik's Cube. *Transactions on Machine Learning Research* (2023). arXiv:2106.03157.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[22] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[23] Daniel Walton. [n. d.]. rubiks-cube-NxNxN-solver. https://github.com/dwalton76/rubiks-cube-NxNxN-solver. Accessed October 2026.